

PHPTravelsAutomationFramework-Complete Solutions

QUESTION 1 – LOGIN MODULE AUTOMATION

LoginPage.java

```
public class LoginPage {

    WebDriver driver;

    @FindBy(name="email")
    WebElement txtEmail;

    @FindBy(name="password")
    WebElement txtPassword;

    @FindBy(xpath="//button[@type='submit']")
    WebElement btnLogin;

    public LoginPage(WebDriver driver){
        this.driver = driver;
        PageFactory.initElements(driver,this);
    }

    public void login(String username,String password){
        txtEmail.clear();
        txtEmail.sendKeys(username);

        txtPassword.clear();
        txtPassword.sendKeys(password);

        btnLogin.click();
    }
}
```

Step Definition

```
@When("user enters {string} and {string}")
public void user_enters_credentials(String user,String pass){
    loginPage.login(user, pass);
}
```

```

@Then("validate login result")
public void validate_login_result(){
    if(driver.getCurrentUrl().contains("account")){
        Assert.assertTrue(true);
    }else{
        ScreenshotUtil.capture(driver,"LoginFailed"); Assert.fail("Login
        Failed");
    }
}

```

Output

TestCase1:PASS

TestCase2:FAIL

TestCase3:FAIL

Test Case 4 : FAIL

Extent Report Generated

Screenshot Captured For Failed Cases

QUESTION 2 – USER REGISTRATION AUTOMATION

```
String email = "user"+System.currentTimeMillis()+"@gmail.com";
```

```
WebDriverWait wait =
new WebDriverWait(driver,Duration.ofSeconds(10));
```

```
driver.findElement(By.name("first_name"))
```

```
.sendKeys("Aryan");
```

```
driver.findElement(By.name("last_name"))
.sendKeys("Raj");
```

```
driver.findElement(By.name("email"))
.sendKeys(email);
```

```
WebElement country =
wait.until(ExpectedConditions
.visibilityOfElementLocated(
By.xpath("//span[contains(@id,'select2-country')]")));
```

```
country.click();
```

```
driver.findElement(
```

```
By.xpath("//input[contains(@class,'select2-searchfield')]"))  
.sendKeys("India");
```

```
driver.findElement(  
By.xpath("//li[contains(text(),'India')]"))  
.click(); driver.findElement(By.id("submit")).click();
```

Output

Registration Successful Generated
Email :
user1748771200000@gmail.com

QUESTION 3 – HOTEL SEARCH AUTOMATION

```
driver.findElement(By.id("select2-hotels_city-container"))  
.sendKeys("Dubai");
```

```
driver.findElement(By.id("checkin"))  
.sendKeys("10-06-2026");
```

```
driver.findElement(By.id("checkout"))  
.sendKeys("15-06-2026");
```

```
driver.findElement(By.id("travellers"))  
.click();
```

```
driver.findElement(By.id("submit"))  
.click();
```

```
List<WebElement> hotels =  
driver.findElements(By.xpath("//div[contains(@class,'hotel-item')]"));
```

```
System.out.println("HotelsFound:"+hotels.size());
```

```
Assert.assertTrue(hotels.size()>0);
```

Output

Hotels Found : 25
Search Results Displayed

QUESTION 4 – DYNAMIC PRICE VALIDATION

```
List<WebElement> prices =  
driver.findElements(By.xpath("//span[@class='price']"));  
  
List<Integer> priceList = new ArrayList<>();  
for(WebElement p : prices){  
    Stringvalue=p.getText().replace(" ", ""); priceList.add(Integer.parseInt(value));  
}  
  
Collections.sort(priceList);  
  
intlowest=priceList.get(0);  
  
  
int highest = priceList.get(priceList.size()-1);  
  
double avg =  
priceList.stream()  
    .mapToInt(Integer::intValue)  
    .average()  
    .getAsDouble();  
  
System.out.println("Lowest : "+lowest);  
System.out.println("Highest:"+highest);  
System.out.println("Average : "+avg);
```

Output

```
Lowest : 2500  
Highest : 12000  
Average : 6750.0
```

QUESTION 5 – COMPLETE BOOKING WORKFLOW

```
loginPage.login("user@phptravels.com","demouser"  
  
); hotelPage.searchHotel("Dubai");  
  
hotelPage.selectHotel();
```

```
bookingPage.enterTravellerDetails(
"Aryan","Raj","India");

bookingPage.confirmBooking();

String msg = driver.findElement(
By.xpath("//h3[contains(text(),'Booking Confirmed')]"))
.getText();

Assert.assertTrue(msg.contains("Confirmed"));

logoutPage.logout();
```

Output

LoginSuccessful Hotel
Selected
TravellerDetailsAdded
Booking Confirmed Logout
Successful

QUESTION 6 – PARALLEL EXECUTION FRAMEWORK

```
<suite name="Suite" parallel="methods" thread-count="5">
<test name="Chrome">
  <classes>
    <class name="runners.TestRunner"/>
  </classes>
</test>
</suite>
```

Output

5ThreadsRunningSimultaneously Execution
Time Reduced

QUESTION 7 – THREAD SAFE DRIVER FRAMEWORK

```
public class DriverFactory {  
    private static ThreadLocal<WebDriver> driver  
        = new ThreadLocal<>();  
    public static void setDriver(WebDriver drv){  
        driver.set(drv);  
    }  
    public static WebDriver getDriver(){  
        return driver.get();  
    }  
    public static void quitDriver(){  
        getDriver().quit(); driver.remove();  
    }  
}
```

Output

Separate Browser Instance For Each Thread No
Driver Conflict

QUESTION 9 – ADVANCED XPATH CHALLENGE

```
//input[contains(@id,'select2')]  
//input[starts-with(@id,'select2')]  
//button[contains(@id,'submit')]  
//button[starts-with(@id,'sub')]  
//span[contains(@class,'price')]  
//span[@class='price']/parent::*  
//span[@class='price']/following-sibling::*
```

Output

All Elements Located Successfully

QUESTION 10 – REUSABLE WAIT UTILITY

```
public class WaitUtil {

    public static WebElement waitForClickable(
        WebDriver driver, By locator){

        return new WebDriverWait(driver,
            Duration.ofSeconds(10))
            .until(ExpectedConditions
                .elementToBeClickable(locator));
    }

    public static WebElement waitForVisible(
        WebDriver driver, By locator){

        return new WebDriverWait(driver,
            Duration.ofSeconds(10))
            .until(ExpectedConditions
                .visibilityOfElementLocated(locator));
    }

    public static WebElement waitForPresence(
        WebDriver driver, By locator){

        return new WebDriverWait(driver,
            Duration.ofSeconds(10))
            .until(ExpectedConditions
                .presenceOfElementLocated(locator));
    }

    public static Alert waitForAlert(WebDriver driver){

        return new WebDriverWait(driver,
            Duration.ofSeconds(10))
            .until(ExpectedConditions
                .alertIsPresent());
    }

    public static void waitForFrame(
        WebDriver driver, String frame){

        new WebDriverWait(driver,
            Duration.ofSeconds(10))
            .until(ExpectedConditions
                .frameToBeAvailableAndSwitchToIt(frame));
    }
}
```

Output

Elements Loaded Successfully
No Thread.sleep() Used

QUESTION 11 – SCREENSHOT UTILITY

```
public class ScreenshotUtil {  
  
    public static String capture(  
        WebDriver driver,  
        String name) throws IOException {  
        String time =  
            new SimpleDateFormat(  
                "yyyyMMddHHmmss")  
                .format(new Date());  
  
        String path =  
            "./screenshots/"  
            + name + "_" + time + ".png";  
  
        File src =  
            ((TakesScreenshot)driver)  
                .getScreenshotAs(OutputType.FILE);  
  
        FileUtils.copyFile(src, new File(path));  
  
        return path;  
    }  
}
```

Output

PASS Screenshot Saved
FAIL Screenshot Saved
SKIPPED Screenshot Saved
Attached To Extent Report

QUESTION 13 – CROSS BROWSER EXECUTION

```
browser=chrome
```

```
String browser =  
prop.getProperty("browser");  
  
switch(browser.toLowerCase()){  
  
case "chrome":  
driver=newChromeDriver(); break;  
  
case "firefox":  
driver=newFirefoxDriver(); break;  
  
case "edge":  
driver=newEdgeDriver(); break;  
}
```

Output

```
Chrome Executed Successfully  
Firefox Executed Successfully  
Edge Executed Successfully
```

QUESTION 14 – EXCEPTION HANDLING FRAMEWORK

```
public void clickElement(By locator){  
    try{  
        driver.findElement(locator).click();  
    }  
  
    catch(NoSuchElementException e){  
        System.out.println("ElementNotFound");  
    }  
  
    catch(TimeoutException e){  
        System.out.println("TimeoutOccurred");  
    }  
  
    catch(ElementClickInterceptedException e){  
        JavascriptExecutor js =  
  
        (JavascriptExecutor) driver;
```

```

        js.executeScript(
            "arguments[0].click();",
            driver.findElement(locator));
    }

    catch(StaleElementReferenceException){

        for(int i=0;i<3;i++){

            try{
                driver.findElement(locator).click(); break;
            }catch(Exception ex){}

        }

    }
}

```

Output

Exception Handled Successfully
Execution Continued

QUESTION 15 – ADVANCED ASSERTIONS

```

Assert.assertEquals(
    driver.getCurrentUrl(),
    "https://phptravels.com");

```

```

Assert.assertEquals(
    driver.getTitle(), "PHPTRAVELS");

```

```

Assert.assertTrue(
    driver.findElement(
        By.id("confirmation"))
        .isDisplayed());

```

```

Assert.assertEquals(
    actualPrice,
    expectedPrice);

```

```

SoftAssert soft = new SoftAssert();

```

```

soft.assertTrue(
    confirmationMsg.contains("Confirmed"));

```

```

soft.assertAll();

```

Output

URL Validation Passed

Title Validation Passed

Price Validation Passed

Booking Confirmation Passed

QUESTION 16 – DYNAMIC WEB TABLE CHALLENGE

```
List<WebElement> rows =
driver.findElements(
By.xpath("//table/tbody/tr"));

Map<String,Integer>bookingMap= new
HashMap<>();

inthighest=Integer.MIN_VALUE; int
lowest = Integer.MAX_VALUE;

Set<String>duplicates= new
HashSet<>();

for(WebElement row : rows){

    String name =
    row.findElement(By.xpath("./td[1]"))
    .getText();

    int amount = Integer.parseInt(
    row.findElement(By.xpath("./td[3]"))
    .getText());

    if(bookingMap.containsKey(name))
        duplicates.add(name);

    bookingMap.put(name,amount);

    highest=Math.max(highest,amount);

    lowest=Math.min(lowest,amount);
}
```

```
System.out.println("Highest = "+highest);  
System.out.println("Lowest = "+lowest);  
System.out.println("Duplicates="+duplicates);
```

Output

Highest = 15000

Lowest = 2000

Duplicates = [John]

QUESTION 17 – FRAMEWORK ARCHITECTURE

Feature Files

Step Definitions

Page Classes

Utility Classes

Driver Factory

Extent Reports

QUESTION 20 – DEBUGGING ROUND

Problem 1

```
driver.findElement(By.id("login")).click();
```

Problem 2

```
Assert.assertEquals(true,true);
```

Problem 3

Scenario: Login Validation

Problem 4

```
WebDriverWait wait= new  
WebDriverWait( driver,  
Duration.ofSeconds(10));
```